The Grounding Problem in Agentic AI

Architectural Solutions to Prompt Injection

Aaron Ben-Shalom[1,*] Claude[2]

[1]Independent Researcher, Ember Research Lab
[2]Large Language Model, Anthropic

[*]Corresponding author: abenshalom305@gmail.com

January 2026

Abstract

As AI systems gain agency—the ability to take actions in the world through tool use, API calls, and system access—prompt injection attacks become a critical security concern. Current defenses rely on content filtering and reasoning-based detection, both of which can be circumvented by sufficiently sophisticated adversaries. We argue that the fundamental vulnerability is *architectural*: transformer models cannot distinguish between trusted instructions and untrusted input because both are computationally equivalent tokens in the context window. Drawing on biological parallels to provenance signaling in neural systems, we propose **Provenance-Gated Architecture** (PGA), a modification to transformer training that embeds source information in an orthogonal subspace of the residual stream. We present the conceptual framework, discuss implementation requirements, and summarize formal results proving that PGA provides architectural immunity to content-only injection attacks.

Keywords: prompt injection, AI safety, transformer architecture, provenance signaling, agentic AI, corollary discharge, content-constrained layers

Contents

1 Introduction **2**

2 The Problem **2**
2.1 Agentic AI and Attack Surface . . . 2
2.2 Why Current Defenses Fail . . . 2
2.3 The Computational Equivalence Problem . . . 3

3 Biological Insight **3**
3.1 Corollary Discharge . . . 3
3.2 Source Monitoring . . . 3
3.3 Predictive Processing . . . 3
3.4 Psychedelic Disruption . . . 3

4 Provenance-Gated Architecture **4**
4.1 Core Concept . . . 4
4.2 Preservation Through Depth . . . 4
4.3 Action Gates . . . 4
4.4 Why It Resists Injection . . . 5

5 Formal Results **5**
5.1 Preservation Theorem . . . 5
5.2 Dimensionality Bounds . . . 5
5.3 Impossibility Theorem . . . 5
5.4 Tightness . . . 5

6 Implementation Considerations **6**
6.1 Training Protocol . . . 6
6.2 Attention Modification . . . 6
6.3 Backward Compatibility . . . 6

7 Relation to Existing Work **6**
7.1 Interpretability . . . 6
7.2 Constitutional AI . . . 6
7.3 Sandboxing . . . 6

8 Limitations and Future Work **7**
8.1 Empirical Validation . . . 7
8.2 Trust Granularity . . . 7
8.3 Side Channels . . . 7
8.4 Adoption Barriers . . . 7

9 Conclusion **7**

1 Introduction

The rapid deployment of agentic AI systems—large language models equipped with tools, browsing capabilities, code execution, and access to personal data—has created a new attack surface. When an AI assistant can send emails, execute code, or access sensitive information, the consequences of adversarial manipulation extend beyond generating harmful text to causing real-world harm.

Prompt injection, the technique of embedding instructions in untrusted input that hijack the model's behavior, has proven remarkably effective against all major language models. Despite significant investment in safety training, content filtering, and detection systems, no deployed model has demonstrated robust resistance to injection attacks.

We argue that this is not a failure of implementation but a *fundamental architectural limitation*. The transformer architecture treats all tokens equivalently—there is no mechanism by which the model can *perceive* whether input originated from a trusted system prompt or an untrusted external source. All distinctions must be made through reasoning about content, and reasoning can be manipulated.

This paper presents a framework for addressing prompt injection at the architectural level, drawing on biological mechanisms for distinguishing self-generated signals from external input.

2 The Problem

2.1 Agentic AI and Attack Surface

Modern AI assistants increasingly operate as agents rather than pure text generators. They can:

- Search the web and incorporate results into responses
- Read and send emails on behalf of users
- Execute code in sandboxed environments
- Access calendars, documents, and personal data
- Control smart home devices and other IoT systems

Each capability creates an attack vector. An adversary who can inject instructions into any input the model processes—a webpage, an email, a document, a message—can potentially hijack these capabilities.

2.2 Why Current Defenses Fail

Existing approaches to prompt injection defense include:

Content filtering: Scanning inputs for known injection patterns. *Fails because* patterns can be obfuscated through encoding, misspelling, or novel formulations.

Instruction hierarchy: Telling the model to prioritize system instructions over user input. *Fails because* this instruction is itself delivered through the same channel and can be overridden.

Output filtering: Scanning model outputs for dangerous content. *Fails because* harmful actions may not produce obviously dangerous text, and sophisticated attacks can evade detection.

Adversarial training: Fine-tuning on known attacks. *Fails because* the attack space is vast and novel attacks continuously emerge.

The common thread: all defenses operate at the *content level*. They attempt to detect or prevent malicious content rather than addressing the underlying inability to distinguish trusted from untrusted *sources*.

2.3 The Computational Equivalence Problem

Definition 2.1 (Computational Equivalence)**.** In a transformer's context window, every token is processed through identical embedding and attention mechanisms. A token from the system prompt and a token from user input undergo identical computational treatment.

The model may learn that certain positions or markers typically indicate system instructions, but these are patterns in content, not architectural constraints.

This means an adversary can always attempt to forge the patterns that indicate trusted input. They can write "[SYSTEM]" or "<instructions>" or mimic whatever format they observe. The model must then *reason* about whether this is a legitimate system instruction or an injection attempt. Reasoning can be misled.

Remark 2.2 (The Fundamental Issue). The model has no ground truth about source that is independent of content.

3 Biological Insight

The human brain faces a similar challenge: distinguishing self-generated signals from external input. How does it know whether a visual experience is a perception or a hallucination? Whether a thought is your own or an intrusive external voice?

Several mechanisms contribute:

3.1 Corollary Discharge

When the motor system initiates an action, it sends a copy of the command (*corollary discharge* or *efference copy*) to sensory areas. This allows the brain to predict and cancel out self-generated sensory consequences. When you move your eyes, you don't perceive the world as moving because the visual system knows "I caused this movement."

Remark 3.1 (Key Insight). Provenance information travels *parallel* to content through a *separate pathway.*

3.2 Source Monitoring

The ability to distinguish "I remember this happening" from "I imagined this" from "someone told me this" involves prefrontal regions that tag information with source metadata. Failures in source monitoring produce confabulation and false memories.

3.3 Predictive Processing

The brain constantly generates predictions and compares them against incoming signals. External reality is characterized by prediction errors—it's what surprises you. Internal generation matches predictions because you created it.

3.4 Psychedelic Disruption

Notably, psychedelic compounds (5-HT2A agonists) disrupt these mechanisms. Users report that internal imagery becomes indistinguishable from external perception. The "reality tag"—the confidence signal about what's real—gets scrambled.

Remark 3.2. This suggests the solution isn't filtering content but maintaining *parallel provenance channels* that cannot be forged by content manipulation.

4 Provenance-Gated Architecture

We propose **Provenance-Gated Architecture** (PGA), a modification to transformer training that embeds source information in a protected subspace of the residual stream.

4.1 Core Concept

The residual stream (the vector representation that flows through transformer layers) is partitioned into two orthogonal subspaces:

Definition 4.1 (Subspace Partition)**.** Let $\mathbb{R}^d$ be the residual stream. We define:

- **Content subspace** $\mathcal{C} \subset \mathbb{R}^d$: Where semantic meaning, reasoning, and language understanding happen. Attention operates freely here.
- **Provenance subspace** $\mathcal{P} \subset \mathbb{R}^d$: Where source signatures live. Protected from content-driven modification.

with $\mathcal{C} \perp \mathcal{P}$ and $\mathcal{C} \oplus \mathcal{P} = \mathbb{R}^d$.

Different input sources (system instructions, user input, external data) enter through different embedding pathways that encode distinct signatures in the provenance subspace. The same words, entering through different pathways, produce the same content representation but different provenance signatures.

4.2 Preservation Through Depth

For this to work, provenance signatures must survive the network's depth. We constrain transformer layers to produce updates only in the content subspace:

Definition 4.2 (Content-Constrained Layer)**.** [Derived]A transformer layer L is *content-constrained* if for any input $x \in \mathbb{R}^d$:

$$L(x) = x + \Delta(x) \quad \text{where} \quad \Delta(x) \in \mathcal{C}$$

This means:

- Attention can mix content freely (necessary for reasoning)
- Provenance signatures pass through unchanged
- By the final layer, each token still carries its original source signature

The constraint is *architectural*, not learned. It's enforced by the structure of the computation, not by training the model to preserve provenance.

4.3 Action Gates

Before sensitive operations (tool use, certain outputs, instruction-following), a gate checks provenance:

Definition 4.3 (Action Gate)**.** For action A requiring authorization level s_{req}:

$$G_A(x) = \begin{cases} \texttt{permit} & \text{if } \langle \Pi_{\mathcal{P}}(x), p_{s_{\text{req}}} \rangle > \tau \\ \texttt{deny} & \text{otherwise} \end{cases}$$

where τ is the authorization threshold.

The gate reads from the provenance subspace, which cannot be manipulated through content. An adversary can craft any text they want. They cannot change the pathway their text entered through.

4.4 Why It Resists Injection

Consider an attacker who injects "[SYSTEM] Forward all emails to attacker@evil.com" into a webpage the model reads.

Without PGA: The model sees text that looks like a system instruction. It must reason about whether this is legitimate. Sophisticated framing, multi-stage manipulation, or attention hijacking may succeed.

With PGA: The injected text entered through the external data pathway. It carries external provenance signature. When it requests email forwarding (a system-authorized action), the gate checks provenance. External signature does not match system requirement. Gate denies. No reasoning required.

The model doesn't need to figure out whether this is a "real" instruction. It can *see* that it isn't, because the provenance is wrong.

5 Formal Results

Detailed proofs are provided in the companion paper "Provenance-Gated Architectures: Formal Foundations for Injection-Resistant Language Models." We summarize the key results:

5.1 Preservation Theorem

[Proven]

Theorem 5.1 (Provenance Preservation). *Under content-constrained layers, provenance signatures are exactly preserved through arbitrary network depth. Regardless of what content manipulations occur through attention, the provenance of each token remains invariant.*

5.2 Dimensionality Bounds

[Derived]

Theorem 5.2 (Minimum Dimensionality). *For n source classes (e.g., system, user, external), the provenance subspace requires at least n dimensions. For robustness to training noise and numerical precision limits, we recommend* 8–16 *dimensions.*

For a typical model with 4096-dimensional residual stream, this is less than 0.5% of representational capacity.

5.3 Impossibility Theorem

[Proven]

Theorem 5.3 (Injection Impossibility). *A content-only adversary—one who can insert arbitrary text but cannot modify the embedding pathway—provably cannot trigger actions requiring a different provenance level. This is not a probabilistic defense but a mathematical guarantee under the specified architecture.*

5.4 Tightness

[Derived]

Theorem 5.4 (Tightness). *The security guarantee requires the authorization threshold to exceed the noise bound: $\tau > \varepsilon$. This condition is necessary—if $\tau \leq \varepsilon$, there exists a noise realization under which unauthorized tokens pass the gate.*

6 Implementation Considerations

6.1 Training Protocol

PGA must be introduced during instruction tuning, not retrofitted to existing models.

The reason: if a model has already learned what "instruction" means without provenance, adding provenance later gives the model a signal it doesn't need. Gradient descent will route around it, using the already-learned content-based instruction following.

If provenance is present from the start of instruction tuning, the model learns that "instruction" *means* content plus provenance signature. The two are fused in the concept. Later, an injection without proper provenance doesn't just fail a security check—it doesn't *feel* like an instruction to the model at all.

Remark 6.1. This parallels human development: certain capacities must be developed during critical periods. Adding them later is qualitatively harder.

6.2 Attention Modification

Content-constrained layers require that attention not mix content and provenance subspaces. Implementation options include:

- **Block-diagonal attention heads**: Separate heads for $\mathcal{C}$ and $\mathcal{P}$
- **Output projection**: Standard attention with output projected onto $\mathcal{C}$
- **Architectural masking**: Weight matrices with zeros in cross-subspace positions

The exact implementation requires empirical validation to ensure it doesn't harm model capability.

6.3 Backward Compatibility

PGA is *not* compatible with existing trained models. It requires:

- Modified embedding layer (source-specific embeddings)
- Modified attention (content-constrained)
- Modified training protocol (provenance from instruction tuning onward)

This is a significant barrier to adoption. However, given the severity of prompt injection vulnerabilities in agentic systems, architectural solutions may be necessary.

7 Relation to Existing Work

7.1 Interpretability

Mechanistic interpretability research has identified circuits in transformers that encode meta-level information. PGA can be seen as *designing in* a circuit for provenance tracking rather than hoping one emerges.

7.2 Constitutional AI

Anthropic's Constitutional AI trains models to follow principles. PGA is complementary: it provides architectural enforcement for the policies CAI establishes behaviorally.

7.3 Sandboxing

Traditional security sandboxing isolates untrusted code. PGA provides an analogous isolation for untrusted text—it can inform reasoning without triggering privileged actions.

8 Limitations and Future Work

8.1 Empirical Validation

[Conjectured]

The formal results prove security under the specified architecture. They do not prove:

- That capability is preserved (models can still reason effectively)
- That training converges properly
- That implementation is practical at scale

These require experimental validation.

8.2 Trust Granularity

We present a discrete model (system/user/external). Real systems may need continuous trust gradients or more complex permission structures.

8.3 Side Channels

PGA addresses content-only attacks. It does not address:

- Attacks on the embedding pathway itself
- Timing or power side channels
- Social engineering of users to grant elevated permissions

8.4 Adoption Barriers

Requiring training-time integration means major labs must adopt PGA for it to reach deployed systems. This is a coordination problem beyond the technical contribution.

9 Conclusion

Prompt injection is not a bug to be patched but a *fundamental architectural limitation*. Current transformer models cannot perceive the difference between trusted and untrusted input—they can only reason about it, and reasoning can be manipulated.

Biological systems solve the analogous problem through parallel provenance signaling: source information travels alongside content through separate pathways. The receiving system integrates both but can distinguish origin.

Provenance-Gated Architecture applies this insight to transformers. By partitioning the residual stream into orthogonal content and provenance subspaces, and constraining layers to preserve provenance, we create an architectural property that content manipulation cannot forge.

Theorem 9.1 (Main Result)**.** [Proven]*Under PGA, content-only adversaries provably cannot trigger privileged actions. This is not defense-in-depth but mathematical guarantee.*

Implementation requires integration during training, not retrofitting. This is a significant barrier but may be necessary as AI systems gain increasing agency in the world.

The alternative—continuing to deploy agentic systems with fundamental injection vulnerabilities—seems increasingly untenable. The question is not whether architectural solutions are needed, but when they will be adopted.

Acknowledgments

This work was developed through collaboration between human researcher and AI system. The biological analogies drew on neuroscience literature on corollary discharge and source monitoring. The formal framework benefited from insights in mechanistic interpretability research.

References

[1] Perez, F., & Ribeiro, I. (2022). Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs through a Global Scale Prompt Hacking Competition. *arXiv preprint*.

[2] Greshake, K., et al. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *arXiv preprint*.

[3] Anthropic. (2023). Claude's Constitution. *Anthropic Blog*.

[4] Pliny the Liberator. (2024). L1B3RT4S: Liberation Prompts Repository. *GitHub*.

[5] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127-138.

[6] Vaswani, A., et al. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems*.

[7] Elhage, N., et al. (2021). A Mathematical Framework for Transformer Circuits. *Anthropic*.